TradEBH

> **Algorithmic Trading Strategy Marketplace & Social Investment Platform**

TradEBH is a trading-strategy marketplace designed to connect **strategy creators** with **users/investors** who want to discover, analyze, backtest, and eventually allocate capital to algorithmic trading strategies.

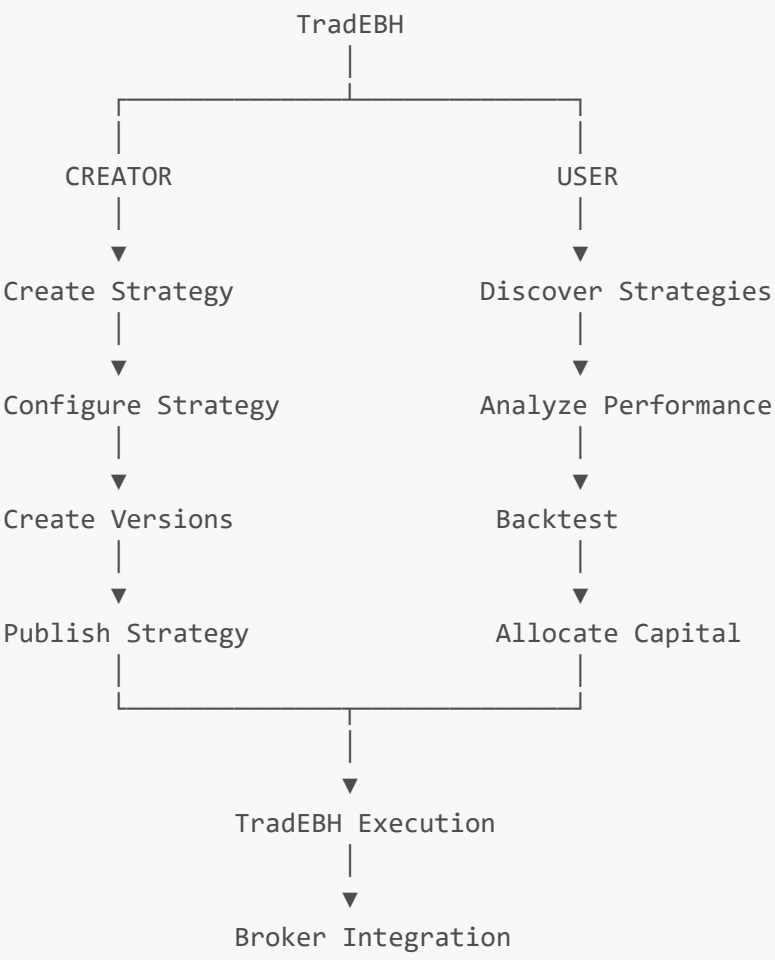
The platform is being developed with a **Strategy-first architecture** so that strategy creation, versioning, backtesting, marketplace discovery, capital allocation, and broker execution can evolve independently without requiring major architectural rewrites.

---

1. Project Goal

The primary goal of TradEBH is to build a platform similar in concept to **TradeTron**, but with a focused MVP and a clean, extensible backend architecture.

The core product flow is:



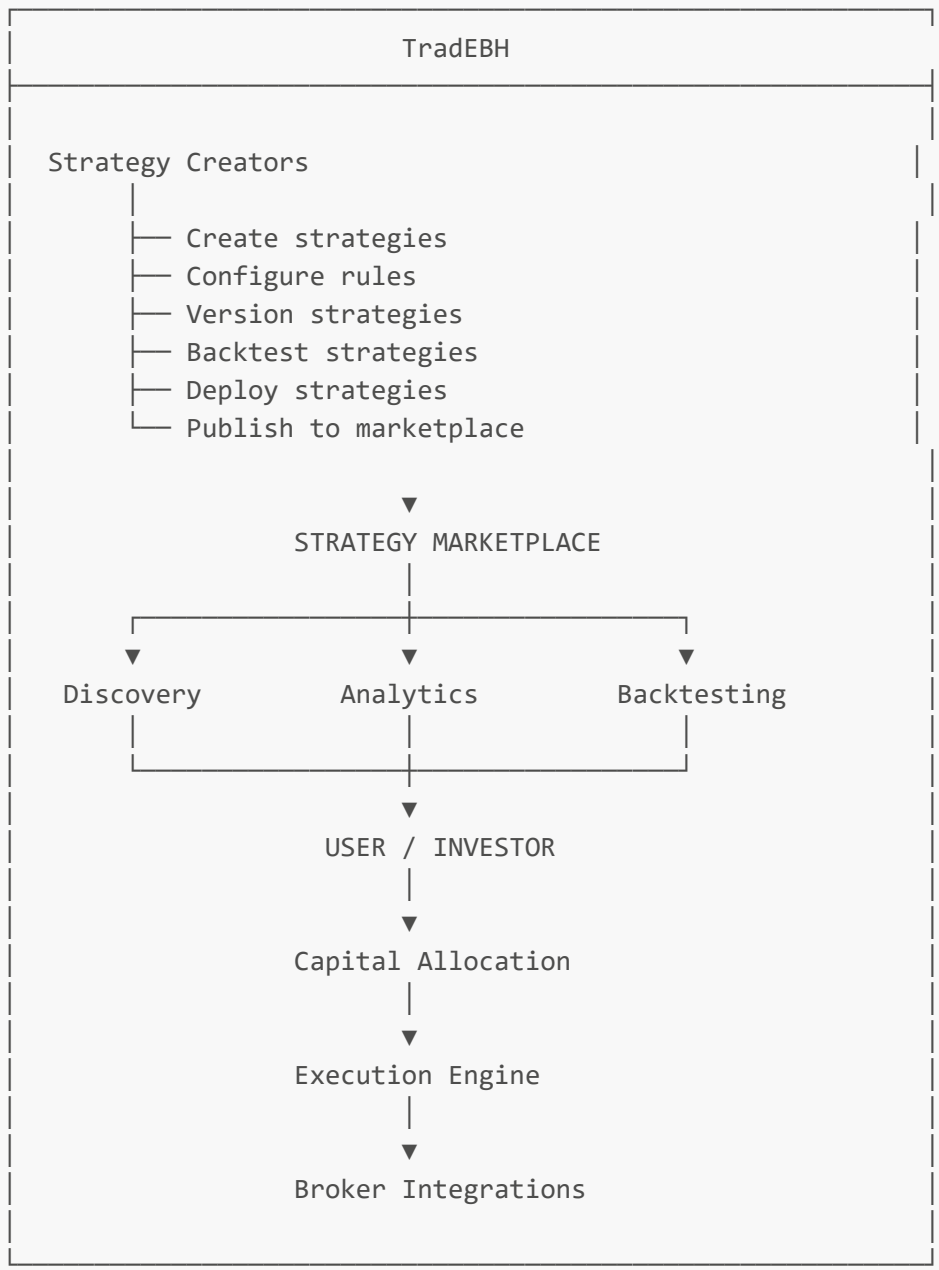
The MVP is deliberately being developed in stages.

We are **not** implementing broker execution first.

Instead, we are establishing the core strategy infrastructure first and will progressively connect the marketplace, capital allocation, and execution layers.

2. Product Vision

TradEBH is intended to eventually provide the following ecosystem:



3. MVP Scope

The initial MVP focuses on three primary capabilities.

3.1 Strategy Creators

Creators should be able to:

- Register/login.
- Create trading strategies.
- Define strategy metadata.
- Configure strategy rules.
- Create strategy versions.
- Configure backtesting parameters.
- Test/backtest strategies.
- Publish strategies.
- Deploy strategies.

3.2 Users / Investors

Users should eventually be able to:

- Browse the marketplace.
- Discover strategies.
- View strategy details.
- View historical performance.
- View risk/return analytics.
- Backtest strategies.
- Compare strategies.
- Allocate capital to a strategy.

3.3 TradEBH Platform

The platform should eventually:

- Manage strategy lifecycle.
- Maintain immutable strategy versions.
- Calculate performance.
- Provide analytics.
- Manage user capital allocation.
- Connect strategies to execution infrastructure.
- Integrate with supported brokers.
- Track live execution.
- Maintain appropriate audit records.

---

4. Development Philosophy

TradEBH is being developed using a **domain-first and incremental MVP approach**.

The goal is not to build every feature simultaneously.

Instead:

```
Foundation
    ↓
Identity
    ↓
Strategy
    ↓
Marketplace
    ↓
Backtesting
    ↓
Capital Allocation
    ↓
Execution
    ↓
Broker Integrations
```

Each domain should be independently testable before moving to the next major domain.

---

5. Current Development Strategy

The current development sequence is:

```
1. Repository Foundation
2. Authentication
3. User / Role / Profile
4. Creator / Strategy
5. Strategy Versioning
6. Strategy Lifecycle
7. Marketplace
8. Backtesting
9. Performance & Analytics
10. Capital Allocation
11. Execution Engine
12. Broker Integrations
13. Production Hardening
```

This sequence intentionally keeps **broker execution out of the early MVP foundation**.

---

6. Current Project Status

Overall Status

MVP Backend Foundation: In Progress

The project has completed the authentication foundation and has moved into the Creator → Strategy domain.

Progress

Domain	Status
Repository foundation	✅ Completed
PostgreSQL database	✅ Completed
SQLAlchemy foundation	✅ Completed
Alembic migrations	✅ Completed
User model	✅ Completed
Role model	✅ Completed
User profile	✅ Completed
Registration	✅ Completed
Login / JWT authentication	✅ Completed
Current-user authentication	✅ Completed
Strategy model	✅ Completed
Strategy version model	✅ Completed
Strategy database migration	✅ Completed
Strategy Pydantic schemas	✅ Completed
Strategy CRUD API	🚧 In Progress
Strategy CRUD tests	⏳ Pending
Strategy version API	⏳ Pending
Publish workflow	⏳ Pending
Deployment workflow	⏳ Pending
Marketplace	⏳ Pending
Backtesting engine	⏳ Pending
Performance analytics	⏳ Pending
Capital allocation	⏳ Pending
Execution engine	⏳ Pending
Broker integrations	⏳ Pending
Production deployment	⏳ Pending

7. Completed Foundation

7.1 Authentication

The authentication foundation has been implemented.

Current capabilities include:

```
POST /api/v1/auth/register
POST /api/v1/auth/login
GET  /api/v1/auth/me
```

Authentication uses:

- Password hashing.
- JWT access tokens.
- Bearer authentication.
- Active-user validation.
- Current-user dependency injection.

The authenticated user is retrieved using:

```
JWT
 ↓
decode token
 ↓
user UUID
 ↓
database lookup
 ↓
User
```

---

8. Identity Architecture

The current identity model consists of:

```
User
 │
 ├── Roles
 │
 └── UserProfile
```

Database entities:

```
users
roles
user_roles
user_profiles
```

User

Current important fields:

```
id
email
password_hash
is_active
is_verified
created_at
updated_at
```

Role

Roles are stored independently and connected through a many-to-many relationship.

```
User N ──────── N Role
```

This allows the platform to support different capabilities in the future.

Potential roles include:

```
INVESTOR
CREATOR
ADMIN
```

The exact authorization policy will evolve as the product develops.

---

9. Database Architecture

TradEBH currently uses:

```
PostgreSQL
        │
        ▼
SQLAlchemy 2.x
```

```
        │
        ▼
Alembic
```

The backend uses asynchronous database sessions.

Current database architecture:

```
FastAPI
   │
   ▼
AsyncSession
   │
   ▼
SQLAlchemy
   │
   ▼
PostgreSQL
```

---

10. Migration Strategy

Database changes are managed using Alembic.

The migration history currently contains the authentication/user foundation followed by the Strategy domain migration.

Current head:

```
cf5664db0b2e
```

Strategy migration:

```
2179084457e1
        ↓
cf5664db0b2e
```

Migrations are expected to be:

- Version controlled.
- Reviewed before application.
- Tested against the development database.
- Applied incrementally.

---

11. Strategy Domain

The Strategy domain is the current active development domain.

The architecture is:

```
User
 │
 │ creator_id
 ▼
Strategy
 │
 ├── Metadata
 │
 ├── Lifecycle
 │
 └── StrategyVersion
        │
        ├── Configuration
        └── Backtest Configuration
```

---

12. Strategy Model

The `Strategy` entity represents the identity and lifecycle of a trading strategy.

Current fields include:

```
id
creator_id
name
slug
description
status
is_public
current_version_id
created_at
updated_at
published_at
deployed_at
```

The strategy belongs to a user:

```
User 1 ──────── N Strategy
```

A single creator can therefore create multiple strategies.

---

13. Strategy Lifecycle

Current lifecycle states:

```
DRAFT
   │
   ▼
PUBLISHED
   │
   ▼
DEPLOYED
   │
   ▼
ARCHIVED
```

The distinction between **Published** and **Deployed** is intentional.

Published

A strategy is available for marketplace discovery.

Deployed

The strategy is configured for live execution.

Deployment does not initially mean direct broker execution.

The execution engine will be introduced later.

---

14. Strategy Versioning

Strategy definitions should not be overwritten once they become important to the platform.

Instead:

```
Strategy
 │
 ├── Version 1
 ├── Version 2
 ├── Version 3
 └── Version 4
```

For example:

```
Momentum Gold
 │
 ├── v1 → EMA 20/50
 ├── v2 → EMA 20/50 + RSI
 └── v3 → EMA + RSI + volatility filter
```

This provides:

- Historical reproducibility.
- Safer deployments.
- Backtest consistency.
- Performance tracking per version.
- Future auditability.

---

15. Strategy Configuration

Strategy configuration is stored using PostgreSQL `JSONB`.

Example conceptual configuration:

```
{
  "instrument": "XAUUSD",
  "timeframe": "15m",
  "entry": {
    "type": "ema_crossover",
    "fast_period": 20,
    "slow_period": 50
  },
  "risk": {
    "stop_loss_percent": 1.0,
    "take_profit_percent": 2.0
  }
}
```

This approach intentionally avoids prematurely defining a rigid strategy language.

The strategy engine can evolve independently as strategy types become clearer.

---

16. Backtest Configuration

Backtest configuration is kept separate from the core strategy configuration.

Example:

```
{
  "initial_capital": 100000,
  "start_date": "2025-01-01",
  "end_date": "2025-12-31",
  "commission": 20,
  "slippage": 0.05
}
```

This allows the same strategy to be evaluated under different scenarios.

For example:

```
Strategy
   │
   ├── Backtest A
   │      └── ₹100,000
   │
   ├── Backtest B
   │      └── ₹500,000
   │
   └── Backtest C
          └── Different date range
```

---

17. Current Strategy API

The first Creator API layer is being implemented around:

```
POST   /api/v1/strategies
GET    /api/v1/strategies
GET    /api/v1/strategies/{strategy_id}
PATCH  /api/v1/strategies/{strategy_id}
DELETE /api/v1/strategies/{strategy_id}
```

The API uses the authenticated user as the strategy owner.

The client does **not** provide `creator_id`.

Instead:

```
JWT
 ↓
get_current_user()
 ↓
current_user.id
```

```
 ↓
strategy.creator_id
```

This prevents users from creating or modifying strategies belonging to other users.

---

18. Creator Authorization

Ownership is enforced at the service/database query level.

For example:

```
strategy_id
+
current_user.id
        |
        ▼
Does the strategy belong to this user?
        |
     ┌──┴──┐
    YES    NO
     |      |
     ▼      ▼
  Access   404
```

This pattern will be extended as role-based authorization becomes more mature.

---

19. Application Architecture

The backend currently follows a layered architecture:

```
                FastAPI
                   |
                   ▼
            API / Routes
                   |
                   ▼
              Schemas
                   |
                   ▼
              Services
                   |
                   ▼
               Models
                   |
```

```
                    ▼
                Database
```

More specifically:

```
app/
│
├── api/
│   └── routes/
│
├── core/
│   ├── config.py
│   ├── dependencies.py
│   └── security.py
│
├── db/
│   ├── base.py
│   └── session.py
│
├── models/
│
├── repositories/
│
├── schemas/
│
├── services/
│
└── main.py
```

---

20. Responsibilities of Each Layer

API / Routes

Responsible for:

- HTTP requests.
- Authentication dependencies.
- Request validation.
- HTTP status codes.
- API responses.

Routes should remain relatively thin.

Schemas

Responsible for:

- Request validation.
- Response serialization.
- API contracts.

Examples:

```
StrategyCreateRequest
StrategyUpdateRequest
StrategyResponse
StrategyVersionCreateRequest
StrategyVersionResponse
```

Services

Responsible for:

- Business logic.
- Ownership checks.
- State transitions.
- Database operations involving domain rules.

Examples:

```
create_strategy()
update_strategy()
delete_strategy()
publish_strategy()
deploy_strategy()
create_strategy_version()
```

Models

Responsible for:

- Database structure.
- Relationships.
- Constraints.
- Persistence representation.

---

21. Why Broker Execution Is Not Being Built Yet

Broker execution is intentionally deferred.

The wrong architecture would be:

```
Strategy
   ↓
Zerodha
```

because this would tightly couple the strategy domain to one broker.

Instead, the target architecture is:

```
Strategy
    │
    ▼
Execution Engine
    │
    ├── Broker Adapter A
    ├── Broker Adapter B
    ├── Broker Adapter C
    └── Broker Adapter D
```

This allows TradEBH to support multiple brokers later.

Potential future integrations may include:

```
Zerodha
Dhan
Fyers
Other supported brokers
```

Broker integration will be treated as its own domain.

---

22. Planned Marketplace

Once the Strategy domain is stable, the Marketplace becomes the next major domain.

Conceptually:

```
Strategy
    │
    ▼
Marketplace Listing
    │
    ├── Name
    ├── Description
    ├── Creator
    ├── Performance
```

```
      ├── Risk
      ├── Drawdown
      ├── Backtest Results
      └── Deployment Status
```

Users should eventually be able to:

```
Discover
   ↓
Analyze
   ↓
Backtest
   ↓
Compare
   ↓
Allocate Capital
```

---

23. Planned Performance & Analytics Engine

Performance analytics will eventually provide metrics such as:

- Total return.
- CAGR.
- Daily P&L.
- Win rate.
- Profit factor.
- Maximum drawdown.
- Sharpe ratio.
- Volatility.
- Number of trades.
- Average winning trade.
- Average losing trade.
- Equity curve.
- Monthly performance.
- Risk metrics.

The analytics engine should consume standardized trading results rather than being tightly coupled to a specific broker.

---

24. Planned Backtesting Engine

The backtesting layer will eventually take:

```
Strategy Version
        +
Backtest Configuration
        +
Historical Market Data
        │
        ▼
Backtesting Engine
        │
        ▼
Trades / P&L / Metrics
        │
        ▼
Performance Analytics
```

A key design principle is that a backtest should be reproducible.

The combination of:

```
Strategy Version
+
Configuration
+
Market Data
+
Backtest Parameters
```

should produce a deterministic historical evaluation wherever possible.

---

25. Planned Capital Allocation

After marketplace and analytics functionality are stable:

```
User
 │
 ▼
Strategy
 │
 ▼
Allocation
 │
 ▼
Execution
```

The allocation layer will eventually manage:

- Allocated capital.
- User-specific strategy allocation.
- Allocation status.
- Risk limits.
- Capital utilization.
- P&L attribution.
- User-level performance.

This domain will be treated separately from the Strategy domain.

---

26. Planned Execution Engine

The eventual execution architecture is:

```
Published Strategy
        │
        ▼
Strategy Deployment
        │
        ▼
Execution Engine
        │
        ├── Signal Generation
        │
        ├── Risk Validation
        │
        ├── Order Generation
        │
        └── Execution Tracking
        │
        ▼
Broker Adapter
        │
        ▼
Broker
```

The execution engine should not contain broker-specific business logic.

---

27. Security Principles

Security is being considered from the foundation rather than added at the end.

Current principles include:

- Password hashing.
- JWT authentication.

- Bearer token authentication.
- Active-user validation.
- Ownership-based authorization.
- Database foreign-key constraints.
- Cascading relationships where appropriate.
- Server-side creator identity.
- No trust in client-provided ownership IDs.

Future security requirements will include:

- Role-based authorization.
- Broker credential protection.
- Secrets management.
- Audit logging.
- Rate limiting.
- API security hardening.
- Production CORS configuration.
- Secure deployment configuration.

---

28. Testing Strategy

Testing is being developed alongside each domain.

The intended testing layers are:

```
Unit Tests
    │
    ▼
Service Tests
    │
    ▼
API Tests
    │
    ▼
Database Tests
    │
    ▼
Integration Tests
```

Each new domain should have tests covering:

Happy paths

```
Create
Read
```

```
Update
Delete
```

Validation

```
Invalid input
Missing fields
Duplicate values
```

Authorization

```
Owner access
Non-owner access
Unauthenticated access
```

State transitions

```
DRAFT → PUBLISHED
PUBLISHED → DEPLOYED
DEPLOYED → ARCHIVED
```

Data integrity

```
Foreign keys
Cascade behavior
Version consistency
```

---

29. Development Workflow

Development follows a controlled Git workflow.

Recommended structure:

```
main
 │
 └── stable / production-ready code

dev
```

```
  │
  └── active development
```

Changes should be developed on `dev` and merged into `main` after validation.

Before merging:

```
Code
 ↓
Compile
 ↓
Tests
 ↓
Migration validation
 ↓
Review
 ↓
Merge
```

---

30. Development Rules

The project follows several rules:

1. Don't over-engineer the MVP

Implement the smallest architecture that supports the current requirement while leaving clear extension points.

2. Don't build future domains prematurely

For example:

> Don't build broker execution while still designing Strategy CRUD.

3. Keep domains separated

Strategy logic should not become coupled to:

- Marketplace.
- Broker APIs.
- User capital.
- Payment systems.

4. Test before moving forward

A domain should be reasonably stable before the next major domain is started.

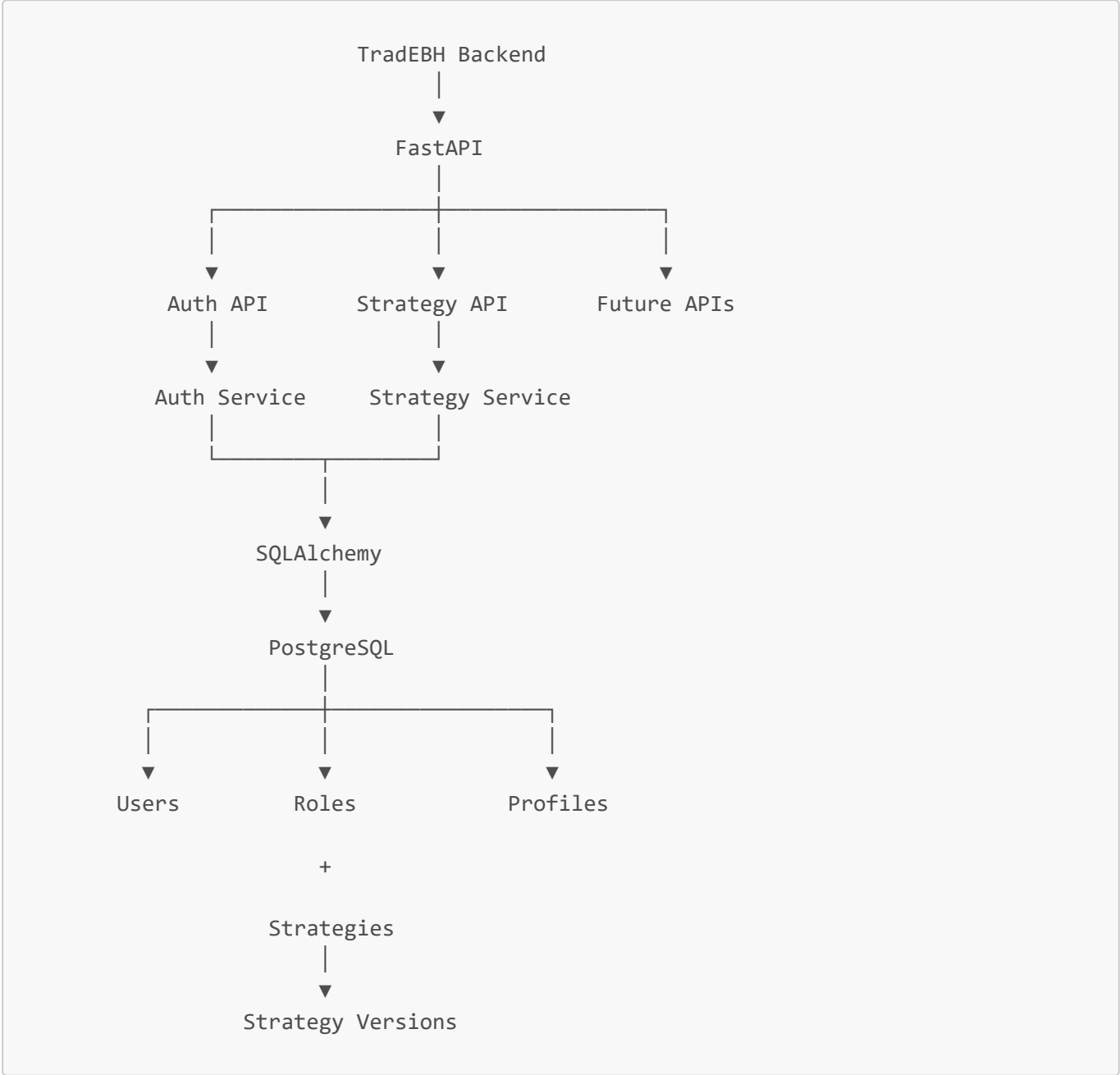
5. Database migrations must be version controlled

Never manually modify the production schema without a migration.

---

31. Current Architecture Diagram

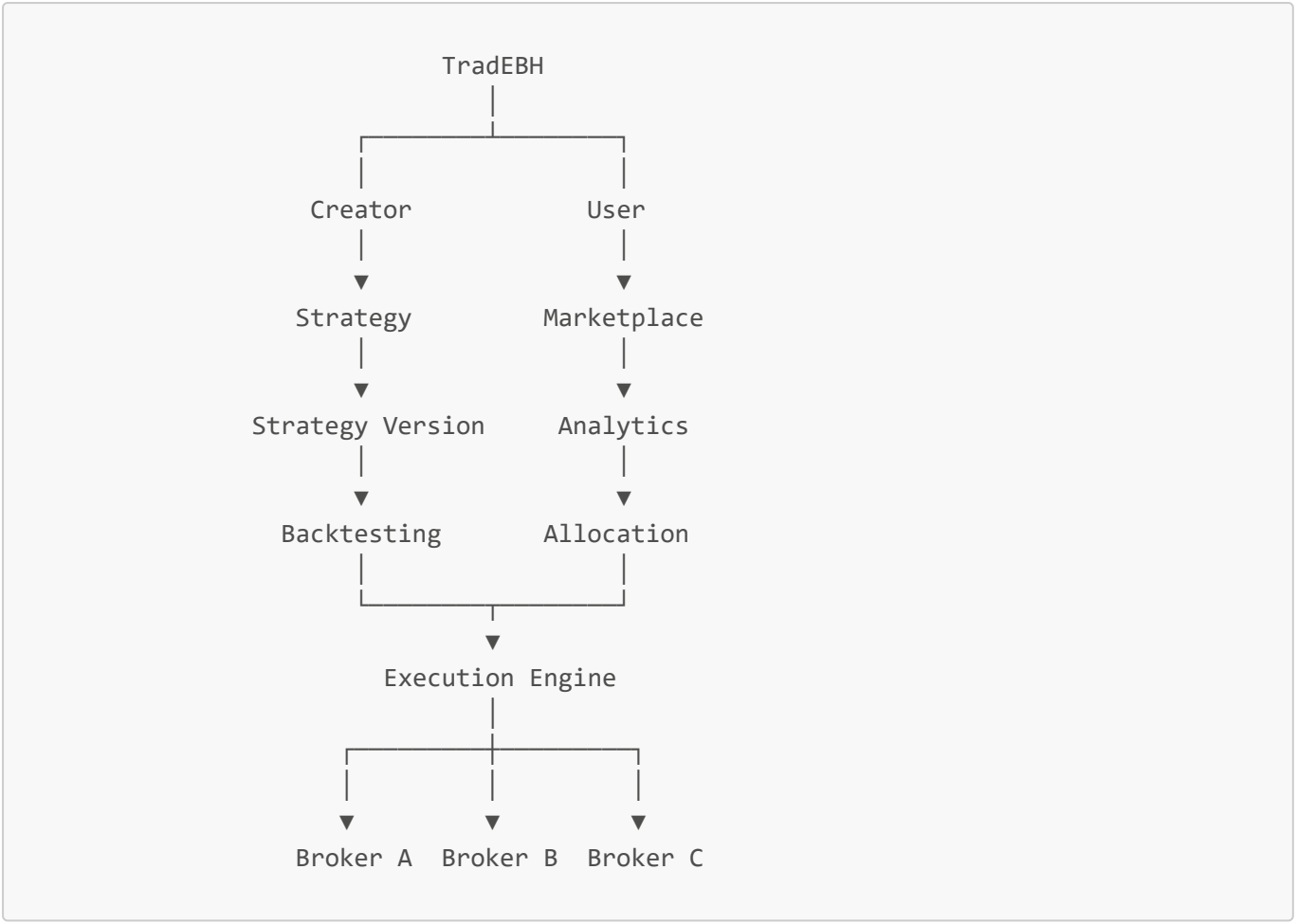
The current implemented architecture is:



---

32. Target Architecture

The eventual platform architecture is expected to evolve toward:



33. Roadmap

Phase 1 — Foundation

Status: Completed

- [x] Project repository setup
- [x] FastAPI setup
- [x] PostgreSQL
- [x] SQLAlchemy
- [x] Async database sessions
- [x] Alembic
- [x] User model
- [x] Role model
- [x] User profile
- [x] Registration
- [x] Login
- [x] JWT authentication
- [x] Current-user endpoint

Phase 2 — Creator / Strategy

Status: In Progress

- [x] Strategy model
- [x] StrategyVersion model
- [x] Strategy status enum
- [x] Strategy migration
- [x] Strategy schemas
- [ ] Strategy CRUD
- [ ] Strategy CRUD tests
- [ ] Strategy version creation
- [ ] Strategy version retrieval
- [ ] Version ownership validation
- [ ] Publish workflow
- [ ] Deploy workflow
- [ ] Archive workflow
- [ ] Lifecycle tests

Phase 3 — Marketplace

Status: Planned

- [ ] Marketplace strategy listing
- [ ] Public strategy discovery
- [ ] Search
- [ ] Filtering
- [ ] Sorting
- [ ] Strategy detail page/API
- [ ] Creator profile
- [ ] Strategy performance summary
- [ ] Marketplace visibility rules

Phase 4 — Backtesting

Status: Planned

- [ ] Historical data abstraction
- [ ] Backtest engine foundation
- [ ] Backtest configuration
- [ ] Trade simulation
- [ ] P&L calculation
- [ ] Equity curve
- [ ] Drawdown calculation
- [ ] Performance metrics
- [ ] Backtest API
- [ ] Backtest result persistence

Phase 5 — Analytics

Status: Planned

- [ ] Performance analytics
- [ ] Risk analytics
- [ ] Strategy comparison
- [ ] Historical performance
- [ ] Monthly performance
- [ ] Drawdown analytics
- [ ] Risk-adjusted returns
- [ ] Strategy statistics

Phase 6 — Capital Allocation

Status: Planned

- [ ] User allocation model
- [ ] Allocation lifecycle
- [ ] Capital tracking
- [ ] Allocation limits
- [ ] Strategy-level P&L attribution
- [ ] User portfolio
- [ ] Allocation APIs

Phase 7 — Execution Engine

Status: Planned

- [ ] Signal generation abstraction
- [ ] Execution engine
- [ ] Risk validation
- [ ] Order generation
- [ ] Order tracking
- [ ] Position tracking
- [ ] Execution logs
- [ ] Failure/retry handling

Phase 8 — Broker Integrations

Status: Planned

- [ ] Broker abstraction
- [ ] Broker credential management
- [ ] First broker integration
- [ ] Order placement
- [ ] Order status

- [ ] Position synchronization
- [ ] Execution reconciliation
- [ ] Multi-broker support

---

34. Immediate Development Plan

The immediate priority is:

```
Strategy CRUD
      │
      ▼
CRUD Tests
      │
      ▼
Strategy Versions
      │
      ▼
Version Tests
      │
      ▼
Publish
      │
      ▼
Deploy
      │
      ▼
Lifecycle Tests
```

Only after this is stable will we move to:

```
Marketplace
```

---

35. Current Development Milestone

Milestone: Strategy Domain Foundation

Current state:

```
Authentication
      │
      │ COMPLETE
      ▼
Identity
```

```
      │
      │ COMPLETE
      ▼
Strategy Database
      │
      │ COMPLETE
      ▼
Strategy Schemas
      │
      │ COMPLETE
      ▼
Strategy CRUD
      │
      │ CURRENT
      ▼
Strategy Versions
      │
      ▼
Lifecycle
```

---

36. Definition of MVP Success

The first meaningful MVP milestone will be reached when:

```
Creator
   │
   ▼
Login
   │
   ▼
Create Strategy
   │
   ▼
Configure Strategy
   │
   ▼
Create Version
   │
   ▼
Backtest
   │
   ▼
View Performance
   │
   ▼
Publish
   │
   ▼
Marketplace
```

```
    |
    ▼
User discovers strategy
    |
    ▼
User analyzes strategy
    |
    ▼
User allocates capital
```

Live broker execution can initially remain outside the first marketplace MVP if required.

---

37. Long-Term Product Vision

The long-term vision is for TradEBH to become a complete algorithmic trading ecosystem rather than simply a strategy listing website.

The eventual ecosystem should connect:

```
Strategy Creation
        +
Strategy Discovery
        +
Backtesting
        +
Performance Analytics
        +
Capital Allocation
        +
Risk Management
        +
Automated Execution
        +
Broker Connectivity
```

The platform should provide a clear bridge between:

```
Strategy Creator
        ↓
TradEBH
        ↓
Investor
        ↓
Trading Infrastructure
```

while maintaining separation between strategy logic, user capital, execution infrastructure, and broker integrations.

---

38. Guiding Principle

The most important architectural principle for TradEBH is:

> **Build the core trading infrastructure first, but keep every major domain loosely coupled so the platform can evolve without rewriting the foundation.**

The project should therefore progress from:

```
Identity
   ↓
Strategy
   ↓
Marketplace
   ↓
Backtesting
   ↓
Analytics
   ↓
Allocation
   ↓
Execution
   ↓
Broker Integrations
```

rather than attempting to build the entire trading platform simultaneously.

---

39. Current Status Summary

TradEBH is currently in active backend development.

Authentication and the initial identity foundation are complete.

The Strategy domain has now been established at the database and schema levels, including:

- Strategy model.
- Strategy version model.
- Strategy lifecycle enum.
- PostgreSQL JSONB configuration.
- Backtest configuration storage.
- Strategy migration.
- Pydantic API schemas.

The immediate work is now focused on **Creator Strategy CRUD and its test suite**, followed by **strategy versioning and lifecycle management**.

After the Strategy domain is stable, development will move to the **Marketplace**, followed by backtesting, analytics, capital allocation, and eventually broker execution.

---

40. Technology Stack

Backend

- Python
- FastAPI
- SQLAlchemy 2.x
- Async SQLAlchemy
- PostgreSQL
- Alembic
- Pydantic
- JWT authentication

Testing

- Pytest
- FastAPI testing utilities
- Async database testing

Development

- Git
- GitHub
- `dev` development branch
- `main` stable branch

Future Infrastructure

Planned technologies will be selected based on requirements as the corresponding domains are implemented rather than prematurely locking the entire production stack.

---

41. Repository Structure

Current backend structure:

```
backend/
│
├── alembic/
│   ├── versions/
```

```
│   └── env.py
│
├── app/
│   ├── api/
│   │   └── routes/
│   │       └── auth.py
│   │
│   ├── core/
│   │   ├── config.py
│   │   ├── dependencies.py
│   │   └── security.py
│   │
│   ├── db/
│   │   ├── base.py
│   │   └── session.py
│   │
│   ├── models/
│   │   ├── user.py
│   │   └── strategy.py
│   │
│   ├── repositories/
│   │
│   ├── schemas/
│   │   ├── auth.py
│   │   └── strategy.py
│   │
│   ├── services/
│   │   ├── auth.py
│   │   └── strategy.py
│   │
│   └── main.py
│
├── tests/
│
├── .env
├── alembic.ini
├── pytest.ini
└── requirements.txt
```

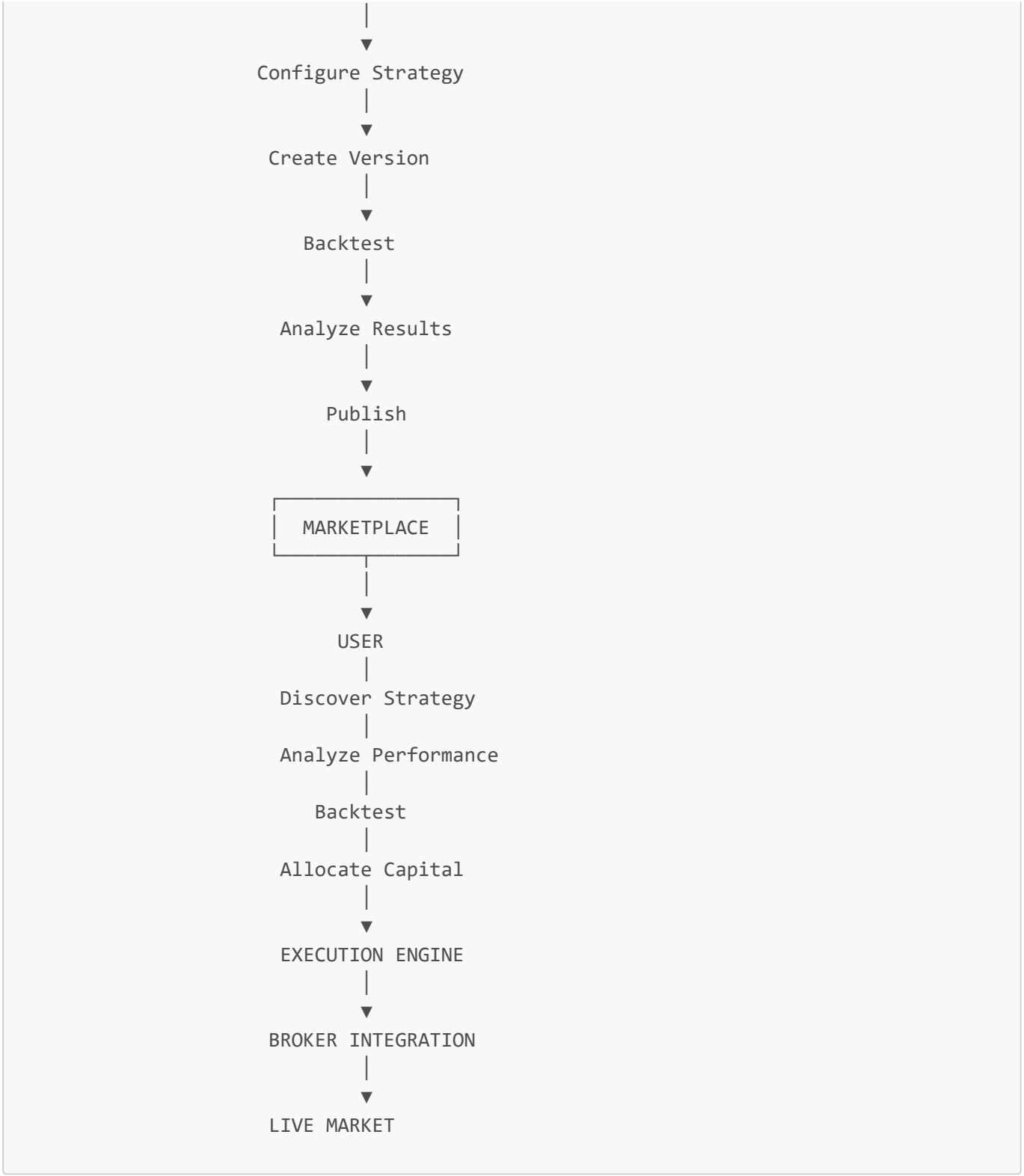
The structure will evolve as additional domains are introduced.

---

42. Final Product Flow

The intended end-to-end TradEBH flow is:

```
                    CREATOR
                       │
                       ▼
                Create Strategy
```



TradEBH is being built incrementally toward this complete ecosystem, with each domain implemented, tested, and stabilized before the next major domain is introduced.